Architecture Specification Document

Project Name: High-Frequency Tick-Data Visualizer (HF-TDV)
Author: Aaish Abbas Zaidi
Institution: JSS University, Noida
Date: April 24, 2026
Document Version: 1.0.0

1. Executive Summary

This document outlines the system architecture for a high-frequency data visualization pipeline capable of rendering 1,000,000 cryptocurrency tick data points per second. The system is designed using a decoupled, microservices approach to ensure continuous data flow from synthetic generation to frontend rendering without causing browser crashes or memory exhaustion.

2. Global Architecture Overview

The system is structured as a strict, unidirectional pipeline. Data generation, buffering, batching, and rendering are isolated into dedicated layers. This prevents bottlenecks, ensuring that no single component waits for downstream processes.

3. Component Specifications

3.1. Layer 1: The Ingestion Engine (Data Generation)

This layer simulates the raw, high-throughput feed of a cryptocurrency exchange order book.

- **Core Technology: Python 3.x**
- **Designation: Data Producer**
- **Operational Mechanism: A high-speed, asynchronous execution loop that generates randomized JSON objects. Each object represents a market tick containing a timestamp, price, and volume.**
- **Performance Mitigation: To bypass standard CPU limitations when generating 1 million unique JSON objects per second, the script utilizes optimized numerical processing libraries (e.g., NumPy) to generate data points in bulk before pushing them to the network.**

3.2. Layer 2: The Message Broker (Buffer & Queue)

This layer acts as the system's shock absorber, preventing the high-velocity ingestion engine from overwhelming the downstream middleware.

- **Core Technology: Apache Kafka & Zookeeper (Containerized via Docker)**
- **Designation: Message Broker**
- **Operational Mechanism: Kafka operates as an independent service maintaining a dedicated topic (crypto-ticks). The Python Producer publishes data directly to this topic. Kafka securely stores these messages in an ordered, highly optimized log.**

- **Fault Tolerance: If the middleware server disconnects, the ingestion engine continues publishing to Kafka uninterrupted. Upon reconnection, the middleware resumes consumption from its last recorded offset, ensuring zero data loss.**

3.3. Layer 3: The Middleware Relay (Batching & Routing)

This layer transforms the raw, continuous stream into optimized payloads suitable for network transmission to a web client.

- **Core Technology: Node.js, kafkajs, WebSockets (ws)**
- **Designation: Kafka Consumer & WebSocket Server**
- **Operational Mechanism:**
  1. **Maintains a continuous connection to Kafka, consuming the crypto-ticks stream.**
  2. **Aggregation Algorithm: Intercepts individual tick objects and aggregates them into an in-memory array.**
  3. **Transmission: At a precise interval of ~16 milliseconds (aligning with a 60Hz monitor refresh rate), the server serializes the aggregated array (potentially containing 16,000+ points) and transmits it via WebSockets to the client, subsequently clearing the array for the next cycle.**

3.4. Layer 4: The Client (Rendering & Memory Management)

The user interface layer, engineered specifically to handle massive dataset injection without DOM or Canvas API degradation.

- **Core Technology: React.js, LightningChart JS**
- **Designation: UI & Visualization Engine**
- **Operational Mechanism:**
  1. **Establishes and maintains a persistent WebSocket connection with the Middleware Relay.**
  2. **Receives high-density arrays of coordinate data and routes them directly to the LightningChart instance.**
  3. **Hardware Acceleration: Utilizes WebGL to bypass the standard browser rendering engine, offloading graphical draw commands directly to the GPU.**
  4. **Memory Optimization (Sliding Window): Enforces a strict X-axis temporal boundary. As new data arrays are rendered, stale data points exiting the visible view-port are actively purged from the browser's memory heap, preventing RAM saturation.**

4. End-to-End Data Flow Summary

1. **Generation: Python script computes {"price": x, "time": y} at scale.**
2. **Buffering: Apache Kafka ingests the payload into the crypto-ticks queue.**
3. **Aggregation: Node.js consumes the queue, aggregating discrete ticks into an array buffer.**
4. **Transmission: Node.js flushes the array buffer down a WebSocket connection every 16ms.**
5. **Rendering: React receives the array payload and feeds it to LightningChart JS.**
6. **Display: GPU renders the updated pixels while simultaneously purging out-of-bounds data points.**